From COCO to the Road: A Reproducible Evaluation of Modern 2D Object Detectors Under Domain Shift

CMPE 249 Intelligent Autonomous Systems · Track A: Experimental Model Evaluation

Advait Shinde · San José State University · Fall 2026

Code, configs and command log: github.com/adv-11/object-detection-for-autonomous-systems

Abstract. We evaluate eight COCO-trained 2D detector configurations from six families — Faster R-CNN, RetinaNet, FCOS, YOLO11 (s, m), RT-DETR and the open-vocabulary YOLO-World and Grounding DINO — under one protocol on COCO val2017 and, without retraining, on the KITTI driving benchmark, and then fine-tune a small detector on KITTI. All eight models reproduce their published COCO AP within 0.9 points. Transferred to KITTI, Car accuracy is essentially preserved (COCO-style AP 49.9 → 50.4 for Faster R-CNN) while Pedestrian and Cyclist lose about two-thirds of their AP, through a mix of taxonomy mismatch, annotation convention and resize-induced shrinkage. The largest zero-shot levers are not architectural: keeping KITTI's wide frames at 1280 px adds 7.6 Moderate mAP to YOLO11m, and the COCO→KITTI Cyclist mapping alone swings Cyclist AP by up to 50 points. YOLO-World-L, sixth on COCO, is the best zero-shot KITTI detector (70.9 Moderate mAP), yet prompting it with KITTI's own class names is far worse than using COCO names and a geometric mapping. Five epochs (8 minutes) of fine-tuning lift YOLO11s from 52.1 to 65.8 Moderate mAP, mostly by fixing localisation at KITTI's 0.7 Car IoU. An adversarial review of the evaluation code found and fixed an ignore-rule bug before any result was reported.

1. Introduction

Detectors are trained and ranked on COCO [1], yet an autonomous vehicle sees a different world: wide 1242×375 frames, objects at distance, heavy occlusion, and a label taxonomy (Car / Pedestrian / Cyclist) that does not match COCO's 80 classes. This report asks whether published COCO accuracy predicts behaviour on a driving benchmark, KITTI [2], when the detector is **not retrained**, and what it takes to close the gap. We evaluate **eight model configurations from six detector families** — two-stage CNN, one-stage anchor-based, one-stage anchor-free, real-time YOLO, real-time DETR and open-vocabulary detectors — under one shared protocol, and answer four questions:

- **RQ1 (reproduction).** Do our numbers match the published COCO results, and how do the families trade accuracy for speed?
- **RQ2 (cross-dataset generalization).** How much accuracy survives the COCO → KITTI shift, and which classes and object scales degrade most?
- **RQ3 (closed-set vs. open-vocabulary).** Can a text-prompted detector use KITTI's own class names instead of a hand-made mapping?
- **RQ4 (fine-tuning).** How much of the gap does a short fine-tune on KITTI train recover?

Before any result was recorded, the evaluation code was put through an adversarial review (Section 3.4) that found and fixed a KITTI ignore-rule bug that would have inflated every KITTI number.

2. Models and datasets

2.1 Detectors

All checkpoints are the official COCO-trained weights (Table 1 lists the reproduced and published accuracy). **Faster R-CNN** [3] with FPN [4] is the two-stage reference; we use torchvision's improved "v2" recipe [5]. **RetinaNet** [6] (v2 weights trained with the ATSS recipe [7]) and **FCOS** [8] represent one-stage dense detectors, anchor-based and anchor-free. **YOLO11** [9], [10] (s and m) is the current Ultralytics real-time CNN, a descendant of YOLO [11]. **RT-DETR-L** [12] is a real-time member of the DETR family [13], [14]: set prediction with no NMS. **YOLO-World-L v2** [15] and **Grounding DINO-T** [16] are open-vocabulary detectors that score boxes against text prompts; neither was trained on COCO annotations, so their COCO numbers are "zero-shot", with the caveat that their grounding data (GoldG) shares images with COCO's sources [15], [16].

2.2 Datasets, splits and taxonomy mapping

COCO val2017 [1] (5,000 images, 80 classes) is the in-domain test. **KITTI 2D object** [2] has 7,481 labelled images; we use the standard split of Chen et al. [17] (3,712 train / 3,769 val, no shared sequences; lists from OpenPCDet [18]). KITTI val is used only for evaluation and is never seen during fine-tuning or model selection. Following the official benchmark [19] we score **Car,**

Pedestrian and Cyclist, and reproduce the devkit's ignore rules: `DontCare` regions absorb any detection that falls inside them (intersection over detection area); `Van` (for Car) and `Person_sitting` (for Pedestrian) are *neighbour classes* that absorb at most one detection matched by IoU; `Truck`, `Tram` and `Misc` are not evaluated, so a Car detection on a truck is a false positive.

KITTI class	Source in a COCO-vocabulary detector	Mismatch
Car	`car`	1:1. COCO has no "van"; KITTI vans are ignore regions for Car.
Pedestrian	`person` that is not riding	COCO `person` also covers riders and sitting people.
Cyclist	`person` + `bicycle` fused into one box	COCO boxes the rider and the bike separately; a KITTI Cyclist box encloses both.
(dropped)	the other 77 COCO classes	`truck`, `bus`, `train` have no evaluated KITTI counterpart.

Table 2. COCO → KITTI taxonomy mapping (src/det_eval/mapping.py).

The Cyclist mapping is the real taxonomy problem. **direct** maps every `bicycle` box to Cyclist — the box covers only the bike and riders become Pedestrian false positives. **fused** (default) pairs a person with a bicycle when they overlap horizontally by ≥50% of the narrower box, the person's top is above the bike's, the person's feet lie in the lower half of the bike's vertical extent, and the person is 0.8–3× the bike's height. The pair becomes one Cyclist with the union box and score $s_{person} \cdot s_{bike}$, the person stays a Pedestrian candidate with score $s_{person} \cdot (1 - s_{bike})$ (a soft re-label that needs no threshold), and unpaired bikes (parked) are dropped. **fused-loose** omits the feet/height tests. The rules were fixed from synthetic cases before any full KITTI evaluation and are reported side by side rather than tuned on val.

3. Experimental setup

3.1 Hardware and software

NVIDIA GeForce RTX 5060 Laptop GPU (Blackwell, 8 GB, **45 W power limit**), Windows 11, Python 3.12, PyTorch 2.11.0 + CUDA 12.8, torchvision 0.26.0 [20], Ultralytics 8.4.197, Transformers 5.22.1, pycocotools 2.0 [21]. Every raw-prediction file stores its full environment, detector configuration and command line.

3.2 Inference protocol

The same protocol is applied to every model so that differences come from the models, not the settings:

- score threshold **0.001**; up to 300 raw detections are kept and the COCO budget of **100 detections per image** is applied *after* mapping to the evaluation taxonomy (otherwise 80-class models would waste slots on classes KITTI ignores);
- each model's native NMS IoU (YOLO 0.7, Faster R-CNN / RetinaNet 0.5, FCOS 0.6) with **multi-label** NMS; none for the DETRs;
- **batch 1, FP32**, images decoded outside the timed region; latency = median per-image time of the full `predict()` call (pre-processing, network, NMS, post-processing) with CUDA synchronisation;
- native input pipelines: torchvision and Grounding DINO resize the short side to 800 px (long side ≤ 1333); YOLO models letterbox the long side to 640 (or 1280) with stride padding; RT-DETR stretches to a 640×640 square.

3.3 Metrics

COCO-style (pycocotools): AP@[.50:.95], AP50, AP75, AP for small ($<32^2$ px), medium and large objects, AR@100 and per-class AP. On KITTI the same metrics are computed for the three classes with every labelled object counted. **KITTI-style**: AP with 40 recall points (AP|R40, the official metric since 2019 [22]) at class-specific IoU (Car 0.7, Pedestrian/Cyclist 0.5) for the Easy / Moderate / Hard levels (min. height 40/25/25 px, max. occlusion 0/1/2, max. truncation 15/30/50 %); objects outside a level become ignore regions and detections shorter than its minimum height are discarded.

3.4 Verification of the evaluation code

Because every number flows through custom mapping and evaluation code, a 47-agent adversarial review (five reviewers covering metric semantics, mapping, detector wrappers, data handling and protocol; two independent refuters per finding, one reproducing on CPU and one reading library source) was run before the final experiments. It upheld 19 of 21 findings. The most important: our first version wrote Van / Person_sitting / difficulty-filtered objects as COCO "crowd" regions, which match by intersection-over-detection and absorb *unlimited* detections, whereas the devkit matches them by IoU, one detection each. Duplicate or partial boxes on vans and occluded cars were silently forgiven, inflating KITTI AP. We now use a `COCOeval`

subclass that honours a separate `kitti_ignore` flag. Other fixes: multi-label NMS for YOLO, applying the detection budget after mapping, and a run-naming collision. Eighteen regression tests in `tests/` encode the reviewers' reproduction cases (e.g. two detections on one van must give AP 50, not 100).

3.5 Commands

```
        uv sync && bash scripts/download_data.sh && uv run python scripts/prepare_kitti.py
        uv run python scripts/run_inference.py --model yolo11m --dataset coco                    # COCO val2017
        uv run python scripts/run_inference.py --model yolo11m --dataset kitti --imgsz 1280      # zero-shot KITTI
        uv run python scripts/run_inference.py --model yolo_world_l --dataset kitti --imgsz 1280 --vocab kitti
        uv run python scripts/finetune_yolo.py --model yolo11s.pt --imgsz 640 --epochs 5 --batch 16
        uv run python scripts/run_inference.py --model yolo11s --dataset kitti --imgsz 640 \
              --weights runs/finetune/yolo11s_kitti_640_e5_f1/weights/last.pt --tag ft
        uv run python scripts/evaluate.py && uv run python scripts/analyze_failures.py <runs> && uv run python scripts/ma
ke_figures.py
        uv run pytest tests
```

Every executed command, with its start time, wall-clock time, throughput and peak memory, is in `logs/command_log.txt`; per-run configurations are in `results/metrics/*.json` (`meta.detector`, `meta.environment`, `meta.command`).

4. Results

4.1 COCO val2017 (in-domain)

Model	Family	Params (M)	AP	Publ. AP	AP50	AP75	APs	APm	APl	AR100	ms/img	Mem (GB)
RT-DETR-L	real-time DETR	32.0	**52.6**	53.0	**70.9**	**57.0**	**33.9**	**57.3**	**70.7**	69.0	61	0.23
YOLO11m	real-time CNN (YOLO)	20.1	51.0	51.5	67.8	55.5	33.3	56.5	67.2	67.3	32	0.18
Grounding DINO-T	open-vocabulary DETR	–	48.3	48.4*	64.0	52.7	33.6	51.5	63.3	**69.6**	464	2.23
Faster R-CNN R50-FPN v2	two-stage CNN	43.7	46.9	46.7	67.9	51.3	30.9	50.8	60.3	63.0	109	0.66
YOLO11s	real-time CNN (YOLO)	9.4	46.1	47.0	62.8	49.9	28.5	50.8	63.5	62.7	29	0.11
YOLO-World-L v2	open-vocabulary CNN	–	45.1	45.1*	60.7	49.2	30.2	49.7	58.0	64.8	27	0.30
RetinaNet R50-FPN v2	one-stage, anchor-based	38.2	41.4	41.5	61.6	43.8	26.8	45.7	53.7	57.8	84	0.37
FCOS R50-FPN	one-stage, anchor-free	32.3	39.2	39.2	58.3	42.0	22.2	42.4	51.3	57.0	81	0.41

Table 1. COCO val2017, 5,000 images. Publ. AP from the official torchvision / Ultralytics documentation and the papers (*zero-shot numbers from [15], [16]). ms/img: median batch-1 FP32 latency of the full predict() call; Mem: peak allocated GPU memory.

RQ1. Every model reproduces its published COCO AP to within 0.9 points (Table 1); torchvision models land within ±0.2 and the open-vocabulary models within 0.1 of their papers. The small, one-directional gap for the Ultralytics models (−0.4 to −0.9) is explained by our protocol: we cap at 100 detections per image and run batch-1 letterboxed inference, whereas the Ultralytics validator keeps 300 detections and uses rectangular batches. RT-DETR-L is the most accurate model (52.6 AP, best on every scale), YOLO11m is 1.6 AP behind at half the latency, and the 2019–2020-era dense detectors (RetinaNet, FCOS) trail by 11–13 AP. On this power-limited laptop GPU the YOLO models are 2.5–4× and RT-DETR-L 1.3–1.8× faster than the torchvision R50-FPN detectors, while Grounding DINO costs ~0.46 s per image, 15× YOLO11m (Figure 1). Absolute latencies are FP32 eager-PyTorch numbers including Python pre/post-processing and are not comparable to the TensorRT FP16 figures in the papers.

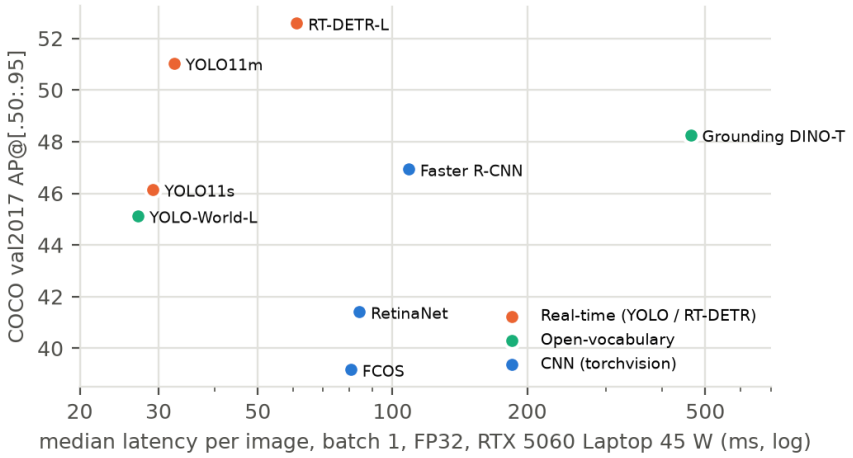


Figure 1. Speed–accuracy on COCO val2017 (batch 1, FP32, 45 W laptop GPU).

4.2 KITTI val without retraining (cross-domain)

Model	Input	AP	AP50	APs	Car	Ped	Cyc	Car	Ped	Cyc	Easy	Mod.	Hard
Faster R-CNN R50-FPN v2	~1333×402	27.2	58.5	20.9	50.4	17.4	13.7	75.3	51.1	49.9	65.2	58.8	50.8
YOLO11s	640×194	25.5	53.5	14.1	48.7	21.2	6.5	72.0	60.0	24.4	63.0	52.1	44.3
YOLO11m	640×194	27.2	58.2	16.4	50.2	22.4	9.1	73.2	64.9	35.0	67.5	57.7	49.5
YOLO11m	1280×387	29.8	64.2	22.4	51.5	24.0	13.8	74.4	70.5	50.9	73.3	65.3	56.1
RT-DETR-L	640×640†	28.9	59.9	19.3	52.8	22.7	11.1	77.0	64.4	40.6	69.2	60.7	52.0
YOLO-World-L (COCO vocab.)	1280×387	**32.1**	**69.4**	**25.4**	**54.2**	**24.9**	**17.2**	**79.7**	**71.6**	**61.5**	**78.9**	**70.9**	**61.7**
YOLO-World-L (KITTI prompts)	1280×387	23.3	46.8	22.1	52.0	10.2	7.8	76.2	42.6	20.5	52.7	46.4	39.8
YOLO11s fine-tuned on KITTI	640×194	38.2	65.0	18.6	65.0	28.0	21.5	86.6	67.0	43.8	77.2	65.8	61.2

Table 3. KITTI val (3,769 images), three classes, default (fused) mapping. Columns 3–8: COCO-style AP@[.50:.95]; columns 9–11: Moderate AP|R40 per class; columns 12–14: mean AP|R40 per difficulty. † RT-DETR stretches the frame to a square. Bold: best zero-shot value. The last row is the fine-tuned model (Section 5.4).

Without retraining, every detector loses roughly half of its COCO-style AP on KITTI (Table 3), but the loss is very unevenly distributed across classes and scales (Section 5.1). The best zero-shot result is YOLO-World-L at 1280 px (70.9 Moderate mAP); the best closed-set one is YOLO11m at 1280 px (65.3). Both remain below what 8 minutes of fine-tuning achieves with a smaller model at a smaller input (Section 5.6).

5. Technical analyses

5.1 Cross-dataset generalization: which classes and scales degrade?

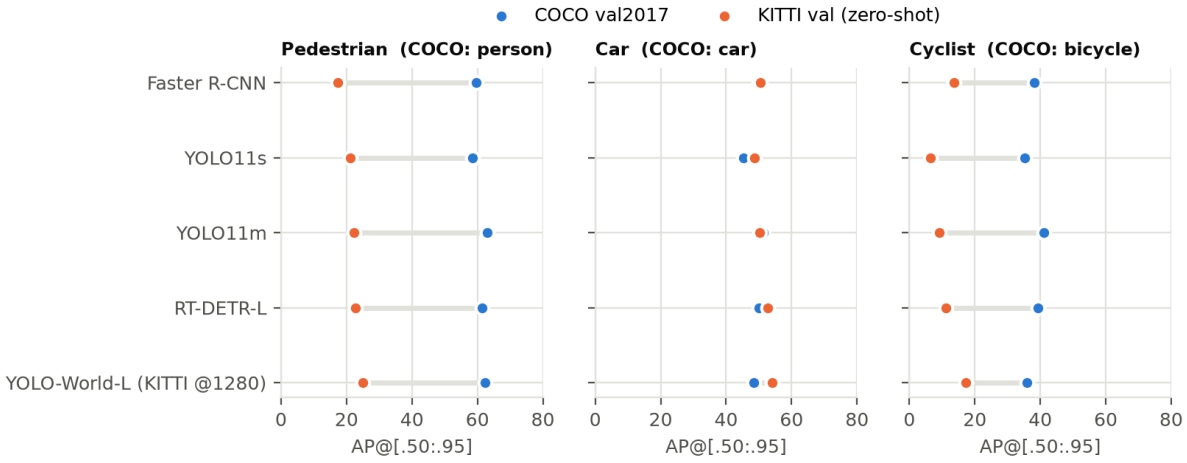


Figure 2. Per-class AP@[.50:.95] on COCO val2017 (blue) and zero-shot on KITTI val (orange). Ultralytics models use 640 px on both datasets; YOLO-World uses 1280 px on KITTI.

Car transfers; people do not. COCO-trained detectors find KITTI cars about as well as COCO cars: Faster R-CNN scores 49.9 AP on COCO `car` and 50.4 on KITTI Car; YOLO11s 45.4 vs. 48.7 (Figure 2). Car AP50 on KITTI is 85–89 for every model. Pedestrian AP, in contrast, falls from ~59–63 on COCO `person` to 17–25 on KITTI, and Cyclist from 35–41 (`bicycle`) to 7–17. Three effects compound for people: (i) **taxonomy** — COCO `person` includes riders and sitting people, so correct COCO behaviour produces KITTI false positives (`confusion:Cyclist` is a major Pedestrian error class in Section 6); (ii) **annotation convention** — KITTI pedestrian boxes are tight, labelled down to small sizes, and heavily occluded instances count, so many detections that would be correct on COCO fail the stricter IoU thresholds (Pedestrian AP50 is 42–61 while Car AP50 is 85–89); (iii) **scale after resizing** — at 640 px the 1242×375 frame is downsampled ×0.52, turning a median 96-px pedestrian into a 50-px one and the tail into sub-20-px objects.

Scale is a resizing artefact, not a property of the data. At native resolution KITTI objects are not smaller than COCO's (median height: Car 50 px vs. 26 px in COCO, Pedestrian 96 px vs. 83 px; Figure 3). They become small only when the frame is shrunk to fit a 640-px input. Running the same YOLO11m weights at 1280 px — no retraining — raises KITTI AP from 27.2 to 29.8, small-object AP from 16.4 to 22.4, and Moderate mAP from 57.7 to 65.3; Cyclist Moderate AP jumps from 35.0 to 50.9 and Pedestrian from 64.9 to 70.5, while Car changes little (73.2 → 74.4) because cars are large enough at either size. On COCO the usual degradation runs from large to small objects (APl 67 → APs 33 for YOLO11m); on KITTI APl is only 36–40 for every model, because "large" KITTI objects are mostly near, truncated cars at the image border, whose boxes COCO models localise poorly at IoU ≥ 0.75.

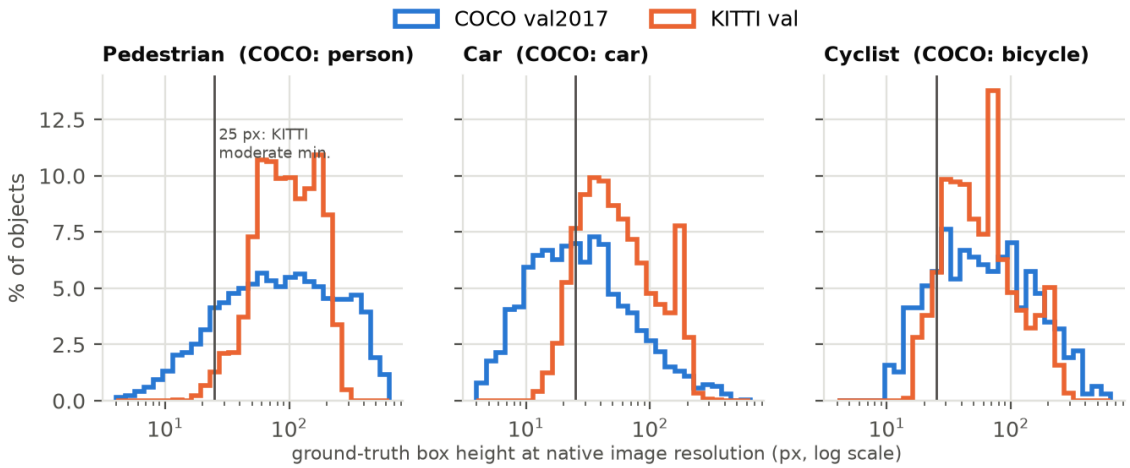


Figure 3. Box heights at native resolution for the three shared classes. KITTI objects are not smaller than COCO's in pixels; downsampling the 1242-px-wide frame to 640 px halves them.

Difficulty. Moderate → Hard costs every zero-shot model 7–9 mAP points (e.g. YOLO11m@1280: 73.3 / 65.3 / 56.1 for Easy / Moderate / Hard), so occlusion and truncation are hard independently of the domain gap.

5.2 Architecture comparison: two-stage vs. one-stage vs. DETR

At the same nominal 640-px setting, the transformer RT-DETR-L generalises best among the closed-set models: 28.9 KITTI AP and 60.7 Moderate mAP versus 27.2 / 57.7 for the CNN YOLO11m, with the best zero-shot Car result (77.0 Moderate) and much higher recall than YOLO11s on sub-25-px objects (Car 79% vs. 64%). Read naively, this says set prediction and global attention transfer better. The input pipelines say otherwise: Ultralytics stretches RT-DETR's input to a 640×640 square, so a KITTI frame is *upsampled* vertically (375 → 640 px) and the model sees 3.3× more pixels than YOLO11m's 640×194 letterbox; tall, thin pedestrians keep their height. Given a comparable pixel budget, the CNN wins — YOLO11m at 1280×387 reaches 65.3 Moderate mAP, 4.6 above RT-DETR-L — so on KITTI the input geometry matters more than the detector family. The two-stage Faster R-CNN at its native ~1333×402 input is competitive overall (58.8 Moderate mAP) and the best zero-shot Cyclist detector (49.9) but the worst on Pedestrian (51.1), where its many confident person boxes are penalised by the mapping (Section 5.3); its region-based box refinement does not help at KITTI's 0.7 Car IoU (19% of its matched cars fall below 0.7, the same as RT-DETR and YOLO11s). All closed-set families converge to AP75 ≈ 22–26 on KITTI despite spanning 46.1–52.6 AP on COCO: the shared COCO training signal, not the architecture, sets the zero-shot ceiling.

5.3 Taxonomy mismatch: how much is the Cyclist mapping worth?

Model	Cyclist mapping	Ped AP	Cyc AP	Ped mod.	Cyc mod.	mAP mod.
Faster R-CNN R50-FPN v2	direct	22.2	0.1	61.7	0.4	45.8
Faster R-CNN R50-FPN v2	fused (loose)	21.1	11.5	62.5	41.0	59.6
Faster R-CNN R50-FPN v2	fused (strict, default)	17.4	13.7	51.1	49.9	58.8
YOLO11s	direct	19.9	0.1	55.6	0.3	42.6
YOLO11s	fused (loose)	21.1	5.8	60.1	20.5	50.8
YOLO11s	fused (strict, default)	21.2	6.5	60.0	24.4	52.1
YOLO11m	direct	20.8	0.1	59.5	0.4	44.4
YOLO11m	fused (loose)	22.0	7.6	64.5	28.1	55.2
YOLO11m	fused (strict, default)	22.4	9.1	64.9	35.0	57.7
YOLO11m	direct	22.1	0.2	64.5	0.4	46.5
YOLO11m	fused (loose)	23.1	11.4	68.6	39.7	60.9
YOLO11m	fused (strict, default)	24.0	13.8	70.5	50.9	65.3
RT-DETR-L	direct	21.5	0.1	60.0	0.1	45.7
RT-DETR-L	fused (loose)	20.7	9.1	60.6	31.2	56.3
RT-DETR-L	fused (strict, default)	22.7	11.1	64.4	40.6	60.7
YOLO-World-L (COCO vocab.)	direct	22.2	0.3	62.5	0.7	47.6
YOLO-World-L (COCO vocab.)	fused (loose)	24.0	15.2	69.5	53.6	67.6
YOLO-World-L (COCO vocab.)	fused (strict, default)	24.9	17.2	71.6	61.5	70.9

Table 4. Sensitivity of Pedestrian/Cyclist accuracy to the Cyclist mapping (COCO-style AP and Moderate AP|R40).

With the naive **direct** mapping, Cyclist Moderate AP is 0.1–0.7 for every COCO-vocabulary model: a bicycle-only box never reaches IoU 0.5 with a KITTI box that also contains the rider. Fusing person and bicycle recovers it to 24–51. The geometric rule then trades the two classes against each other: the **strict** rule (default) gives the best Cyclist AP for every detector, while its duplicate suppression lowers the scores of people standing next to bikes, which costs Faster R-CNN 11 Pedestrian points but YOLO11 almost nothing (its multi-label NMS already merges overlapping person boxes). Swings of up to 50 AP for closed-set models (61 for YOLO-World) on one class from a few lines of post-processing show that cross-taxonomy numbers are only meaningful together with the mapping that produced them.

5.4 Closed-set vs. open-vocabulary

Detector and vocabulary	Car AP	Ped AP	Cyc AP	Car mod.	Ped mod.	Cyc mod.	mAP mod.
YOLO-World-L: 80 COCO names + mapping	54.2	24.9	17.2	79.7	71.6	61.5	70.9
YOLO-World-L: KITTI names (car, pedestrian, cyclist, ...)	52.0	10.2	7.8	76.2	42.6	20.5	46.4
YOLO-World-L: KITTI names, 'person riding a bicycle'	52.0	10.1	3.6	76.1	42.5	9.8	42.8

Table 5. Open-vocabulary detectors on KITTI val with different prompt vocabularies (COCO-style AP and Moderate AP|R40).

Open-vocabulary pre-training generalises best — but not through the prompts. YOLO-World-L, *queried with the 80 COCO names and the same COCO→KITTI mapping as the closed-set models*, is the strongest zero-shot detector in the study: 32.1 KITTI AP and 70.9 Moderate mAP (Car 79.7, Pedestrian 71.6, Cyclist 61.5), ahead of YOLO11m at the same 1280-px input (65.3) even though it is 5.9 AP *worse* on COCO (45.1 vs. 51.0). Its region-text pre-training on Objects365, GoldG and pseudo-labelled CC3M [15] covers far more scenes, scales and viewpoints than COCO alone, and that breadth transfers to the road better than in-domain COCO accuracy does. In-domain rank is therefore a poor predictor of out-of-domain rank.

Prompting with KITTI's own class names makes it worse. Replacing the COCO vocabulary by KITTI's names drops Pedestrian from 71.6 to 42.6 and Cyclist from 61.5 to 20.5 (Table 5); the descriptive phrase "person riding a bicycle" is worse still (Cyclist 9.8). Car, whose name is identical in both vocabularies, barely moves (79.7 → 76.2). "Pedestrian" and "cyclist" are rare words in image-text data compared with "person" and "bicycle", so their text embeddings are poorly grounded, and a phrase describes a relation (person *on* bicycle) that a region-text matcher scores on the person region alone, so the predicted box misses the bike. The promise of open-vocabulary detection — "just name the target classes" — did not hold here: the closed-set-style vocabulary plus an explicit geometric mapping beat every KITTI-native prompt.

5.5 Speed–accuracy trade-off

On COCO the Pareto front on this GPU is YOLO-World-L (27 ms, 45.1 AP) → YOLO11s (29 ms, 46.1) → YOLO11m (32 ms, 51.0) → RT-DETR-L (61 ms, 52.6); the torchvision detectors are dominated (81–109 ms for 39–47 AP) and Grounding DINO pays 464 ms for open-vocabulary flexibility at 48.3 AP. On KITTI, input resolution becomes the main speed–accuracy knob: YOLO11m at 1280 px gains 7.6 Moderate mAP over 640 px, more than the gap between YOLO11s and YOLO11m at the same resolution (5.6). Peak GPU memory is small for every real-time model (0.1–0.3 GB at batch 1) and largest for Grounding DINO (2.2 GB), so on an 8-GB GPU memory is not the binding constraint for inference — it was for training, where YOLO11m at 1280 px did not fit and the fine-tune had to use YOLO11s.

5.6 Fine-tuning: zero-shot vs. adapted

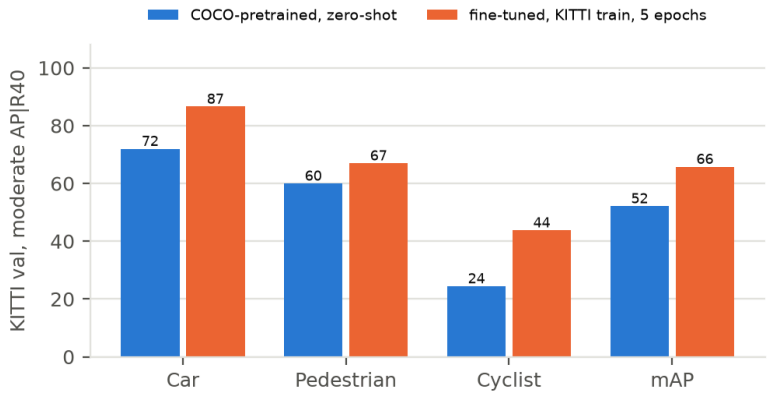


Figure 4. YOLO11s at 640 px: COCO weights zero-shot vs. after 5 epochs (8 minutes) of fine-tuning on KITTI train. KITTI val Moderate AP|R40.

Five epochs on KITTI train (3,712 images, 8 minutes, 4.1 GB) lift YOLO11s from 25.5 to 38.2 COCO-style AP and from 52.1 to 65.8 Moderate mAP (Figure 4). Car gains the most (72.0 → 86.6 Moderate) even though it "transferred" zero-shot: the gain is mostly localisation — the share of matched cars whose IoU falls below KITTI's 0.7 threshold drops from 21% to 9% (Section 6). Cyclist nearly doubles (24.4 → 43.8) because the model now predicts KITTI's rider-plus-bike box directly. Pedestrian gains least

(60.0 → 67.0): with ~2,200 training pedestrians and 5 epochs at 640 px, recall on pedestrians shorter than 40 px actually falls. The 5-epoch, 640-px YOLO11s already matches YOLO11m zero-shot at 1280 px (65.8 vs. 65.3 Moderate mAP) at roughly half the latency (22 vs. 45 ms): **for a fixed deployment budget, a few minutes of in-domain fine-tuning are worth more than a larger model or a larger input.**

6. Failure analysis

Model	Class	<25	25–40	40–60	60–100	>100	occ 0	occ 1	occ 2	Car IoU<0.7 (%)
Faster R-CNN	Car	80	77	69	71	86	96	78	35	19
Faster R-CNN	Pedestrian	75	83	75	75	91	93	86	50	
RT-DETR-L@640	Car	79	75	69	70	85	95	77	34	19
RT-DETR-L@640	Pedestrian	84	63	77	74	86	91	82	42	
YOLO11s@640 zero-shot	Car	64	71	68	71	85	92	70	33	21
YOLO11s@640 zero-shot	Pedestrian	41	46	73	75	87	90	80	41	
YOLO11s@640 fine-tuned	Car	69	86	92	94	97	95	85	80	9
YOLO11s@640 fine-tuned	Pedestrian	3	20	62	77	95	88	77	52	

Table 6. Recall (all detections, class IoU threshold) by ground-truth box height (px) and occlusion level, and share of matched Car detections (score ≥ 0.3, IoU ≥ 0.5) whose IoU is below 0.7.

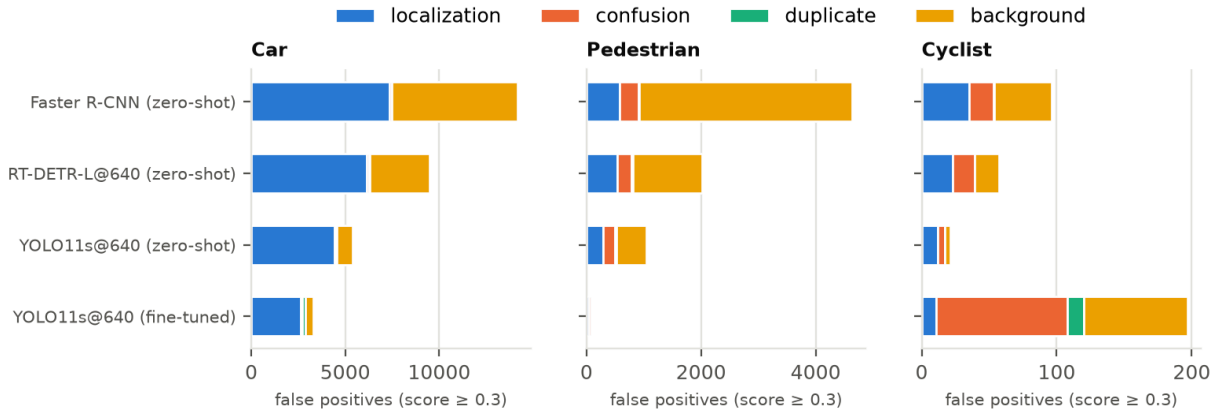


Figure 5. False positives at score ≥ 0.3, classified TIDE-style [23] against the full KITTI label set: localisation (same-class GT with 0.1 ≤ IoU < threshold), confusion (IoU ≥ 0.5 with a GT of another class), duplicate, background. Detections absorbed by ignore regions are excluded.

Localisation, not detection, limits Car. Zero-shot detectors find 85–86% of cars taller than 100 px, yet 19–21% of the cars they match at IoU 0.5 fall short of 0.7, and localisation errors are the largest Car false-positive class for the YOLO models (Figure 5). COCO's box convention for partially visible and truncated vehicles differs from KITTI's, and the strict 0.7 IoU turns that into misses. Fine-tuning halves the problem (9% below 0.7).

Small and occluded people are missed; riders are double-counted. For the 640-px YOLO models, Pedestrian and Cyclist recall falls steeply below 40 px (YOLO11s zero-shot: Cyclist 2% below 25 px vs. 85% above 100 px), while the higher-resolution inputs of Faster R-CNN and RT-DETR keep small-pedestrian recall near their large-object level, and heavy occlusion roughly halves Pedestrian recall. Among Pedestrian false positives, `confusion:Cyclist` (a rider detected as a pedestrian) is the third-largest class after background and localisation, a direct symptom of the taxonomy gap; after fine-tuning the dominant Cyclist error flips to the opposite confusion (pedestrians walking next to bikes called cyclists). Background false positives (poles, signs, shop fronts detected as people) are several times more frequent for Faster R-CNN than for YOLO11 at the same 0.3 operating point, reflecting different score calibration rather than worse ranking.



Figure 6. KITTI val examples (score ≥ 0.3). Blue: Car, orange: Pedestrian, aqua: Cyclist; grey dotted: ignore regions. Left to right: ground truth, YOLO11s@640 zero-shot, fine-tuned. Zero-shot misses the distant cyclist (row 1) or calls it a pedestrian (row 3) and misses far, small cars; the fine-tuned model recovers both, but still splits the occluded pedestrian at the left edge of row 2.

7. Limitations

The experiment was time-boxed to a single laptop GPU, which shaped several choices. Fine-tuning is a single seed, 5 epochs, 640 px and one model (YOLO11s), so the fine-tuning gains are a lower bound; 1280-px training of YOLO11m did not fit in 8 GB. Grounding DINO was run on KITTI with one vocabulary, and RetinaNet, FCOS and RT-DETR at 1280 px were not run on KITTI. Latency is batch-1 FP32 eager PyTorch with Python overhead on a 45-W GPU and should be read as relative, not absolute. KITTI AP is computed with pycocotools (40-point recall grid, devkit ignore rules reproduced and unit-tested) but was not cross-checked against the official C++ binary on full predictions. The "zero-shot" COCO results of the open-vocabulary models carry the image-overlap caveat of their grounding data.

8. Conclusions

1. **Published COCO accuracy is reproducible** under one uniform protocol (all eight models within 0.9 AP); on COCO the real-time models dominate — RT-DETR-L is the most accurate, YOLO11m the best accuracy per millisecond.
2. **COCO accuracy predicts KITTI Car accuracy but not Pedestrian or Cyclist.** Car AP is essentially unchanged across datasets, while people lose about two-thirds of their AP to taxonomy, box-convention and resize effects.
3. **The biggest zero-shot levers are the input pipeline and the label mapping, not the architecture:** keeping KITTI at 1280 px adds 7.6 Moderate mAP to YOLO11m, and the Cyclist mapping alone swings Cyclist AP by up to 50 points.
4. **Broad open-vocabulary pre-training transfers better than COCO accuracy, but prompts are not a substitute for a mapping:** YOLO-World-L is the best zero-shot KITTI detector (70.9 Moderate mAP) despite ranking sixth on COCO, yet prompting it with KITTI's own class names loses 41 Cyclist AP (52 with a descriptive phrase) relative to COCO names plus a geometric mapping.
5. **A few minutes of fine-tuning beat everything else:** 5 epochs on KITTI train lift YOLO11s by 13.7 Moderate mAP, mostly by fixing localisation at KITTI's 0.7 IoU, making a small 640-px model as accurate as a larger 1280-px zero-shot one.
6. **Evaluation code deserves the same scrutiny as models:** an adversarial review caught an ignore-rule bug that would have inflated every KITTI number before any result was reported.

References

1. T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in *Proc. ECCV*, 2014, pp. 740–755. arXiv:1405.0312.
2. A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *Proc. IEEE CVPR*, 2012, pp. 3354–3361.
3. S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *Advances in NIPS 28*, 2015. arXiv:1506.01497.
4. T.-Y. Lin et al., "Feature Pyramid Networks for Object Detection," in *Proc. IEEE CVPR*, 2017. arXiv:1612.03144.
5. Y. Li et al., "Benchmarking Detection Transfer Learning with Vision Transformers," arXiv:2111.11429, 2021.
6. T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE ICCV*, 2017. arXiv:1708.02002.

7. S. Zhang et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *Proc. IEEE/CVF CVPR*, 2020. arXiv:1912.02424.
8. Z. Tian, C. Shen, H. Chen, and T. He, “FCOS: Fully Convolutional One-Stage Object Detection,” in *Proc. IEEE/CVF ICCV*, 2019. arXiv:1904.01355.
9. G. Jocher and J. Qiu, “Ultralytics YOLO11,” software v11.0.0, 2024. Docs: https://docs.ultralytics.com/models/yolo11/; code: https://github.com/ultralytics/ultralytics.
10. R. Khanam and M. Hussain, “YOLOv11: An Overview of the Key Architectural Enhancements,” arXiv:2410.17725, 2024.
11. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016. arXiv:1506.02640.
12. Y. Zhao et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *Proc. IEEE/CVF CVPR*, 2024, pp. 16965–16974. arXiv:2304.08069.
13. N. Carion et al., “End-to-End Object Detection with Transformers,” in *Proc. ECCV*, 2020. arXiv:2005.12872.
14. X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *Proc. ICLR*, 2021. arXiv:2010.04159.
15. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-World: Real-Time Open-Vocabulary Object Detection,” in *Proc. IEEE/CVF CVPR*, 2024. arXiv:2401.17270.
16. S. Liu et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *Proc. ECCV*, 2024. arXiv:2303.05499.
17. X. Chen et al., “3D Object Proposals for Accurate Object Class Detection,” in *Advances in NIPS 28*, 2015.
18. OpenPCDet Development Team, “OpenPCDet: An Open-source Toolbox for 3D Object Detection from Point Clouds,” 2020. https://github.com/open-mmlab/OpenPCDet.
19. A. Geiger, P. Lenz, and R. Urtasun, “KITTI 2D Object Detection Benchmark and devkit,” https://www.cvlibs.net/datasets/kitti/eval_object.php?obj_benchmark=2d (accessed Nov. 2026).
20. TorchVision maintainers and contributors, “TorchVision: PyTorch’s Computer Vision Library,” 2016; detection model docs: https://docs.pytorch.org/vision/stable/models.html.
21. COCO Consortium, “COCO API (pycocotools),” https://github.com/cocodataset/cocoapi; “Detection evaluation,” https://cocodataset.org/#detection-eval.
22. A. Simonelli, S. Rota Bulò, L. Porzi, M. López-Antequera, and P. Kontschieder, “Disentangling Monocular 3D Object Detection,” in *Proc. IEEE/CVF ICCV*, 2019. arXiv:1905.12365.
23. D. Bolya, S. Foley, J. Hays, and J. Hoffman, “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *Proc. ECCV*, 2020. arXiv:2008.08115.
24. Y. Wang et al., “Train in Germany, Test in the USA: Making 3D Object Detectors Generalize,” in *Proc. IEEE/CVF CVPR*, 2020. arXiv:2005.08139.
25. Hugging Face, “Grounding DINO — Transformers documentation,” https://huggingface.co/docs/transformers/model_doc/grounding-dino; checkpoint IDEA-Research/grounding-dino-tiny.